Assignment No: 9

Title: Develop a ReactJS application demonstrating props, state, hooks, event handling, and conditional rendering.

Aim: To develop a ReactJS application that demonstrates passing data with props, managing local state, using the useState and useEffect hooks, writing a custom hook, handling events, and rendering different output conditionally.

Objectives:

1. To pass read-only data from a parent component to a child component using props.
2. To hold and update component-local data using the useState hook.
3. To run side effects and subscribe or unsubscribe from external sources using useEffect.
4. To extract reusable stateful logic into a custom hook.
5. To handle user events and update state from the handler.
6. To render different JSX conditionally using &&, the ternary operator and early return.
7. To lift state up so that sibling components stay synchronized.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- Node.js (v18 or later) and npm
- Vite build tool, React 19
- Firebase JavaScript SDK (for the live data subscription)

Theory:

1. Props

Props (properties) are the inputs of a component. They flow one way, from parent to child, and are read-only inside the child: a component must never assign to its own props. To let a child affect the parent, the parent passes down a callback function as a prop.

Example:

```
// parent
<ResultCard result={result} theme={theme} />

// child, with destructuring
function ResultCard({ result, theme }) { ... }
```

2. State and the useState Hook

State is data owned by a component that may change over time. useState returns the current value and a setter; calling the setter schedules a re-render. State must never be mutated directly - a new value or a new object must be supplied.

Example:

```
const [lang, setLang] = useState("hi");          // initial value
const [mode, setMode] = useState(() =>             // lazy initialiser
  localStorage.getItem("themePreference") || "DARK"
);

setMode(prev => prev === "DARK" ? "LIGHT" : "DARK");  // functional update
```

3. The useEffect Hook

useEffect runs code after the component has rendered. The second argument is the dependency array, which controls when the effect runs again. If the effect starts a subscription, a timer or a listener, it must return a cleanup function that stops it, otherwise the application leaks.

Dependency array	The effect runs
omitted	after every render
[]	once, after the first render
[a, b]	after the first render and whenever a or b changes

4. Rules of Hooks and Custom Hooks

- Hooks may only be called at the top level of a component, never inside a condition, a loop or a nested function.
- Hooks may only be called from a React function component or from another hook.
- A custom hook is a function whose name begins with 'use' and which calls other hooks; it packages reusable stateful logic.

5. Event Handling

React wraps native events in a cross-browser SyntheticEvent and uses camelCase handler props such as onClick, onChange, onSubmit, onDrop and onKeyDown. A handler is passed as a function reference, not called: onClick={handleClick} or onClick={() => handleClick(id)}. e.preventDefault() suppresses the browser's default behaviour.

6. Conditional Rendering

Technique	Use when
{cond && <X/>}	render something or nothing
{cond ? <X/> : <Y/>}	choose between two outputs
if (!data) return null;	a guard clause at the top of the component
{list.length === 0 ? <Empty/> : list.map(...)}	empty-state handling

7. Lifting State Up

When two sibling components need the same value, that value is moved into their closest common parent and passed down as props, together with the setter. In LipiSutra, PublicView owns the result state and passes setResult to UploadSection as the onResult prop, so the upload component can feed the result back up to its parent.

Course Project Implementation:

Course-Project Mapping: Fully available in the course project. Submit src/components/MuseumView.jsx as the primary file - it contains a hand-written custom hook (useCountUp), useState, useEffect with a cleanup function, props, conditional rendering and list rendering - and submit src/components/PublicView.jsx alongside it for event handling, tabbed conditional rendering and lifting state up.

Concept	File	Evidence
props (object + callback)	src/components/PublicView.jsx	theme prop in; setImage / setLoading / onResult passed down to UploadSection
useState	src/components/PublicView.jsx	eight state variables: image, base64, result, loading, lang, translation, searchQuery, activeTab
useEffect + cleanup	src/components/MuseumView.jsx	onSnapshot subscription returned as the cleanup function
custom hook	src/components/MuseumView.jsx	function useCountUp(target, duration)
event handling	src/components/UploadSection.jsx	onDrop, onDragOver, onChange, onClick
conditional rendering	src/components/PublicView.jsx	{!image && !result && (...)}, {loading && (...)}, {activeTab === 'summary' && (...)}
lifting state up	src/components/PublicView.jsx	result lives in the parent, UploadSection calls onResult(geminiResult)

Program:

Project file - src/components/MuseumView.jsx (custom hook + useEffect + cleanup)

```
import { useState, useEffect } from "react";
import { db } from "../services/firebase";
import { collection, onSnapshot, orderBy, query } from "firebase/firestore";

/* ---------- CUSTOM HOOK : animates a number from 0 up to `target` ---------- */
function useCountUp(target, duration = 1200) {
  const [value, setValue] = useState(0);

  useEffect(() => {
    if (!target) { setValue(0); return; }

    let current = 0;
    const step  = Math.ceil(target / (duration / 16));
```

```
    const timer = setInterval(() => {
      current += step;
      if (current >= target) { current = target; clearInterval(timer); }
      setValue(current);
    }, 16);

    return () => clearInterval(timer);      // CLEANUP : stop the timer
  }, [target, duration]);                   // DEPENDENCIES

  return value;
}

/* ---------- COMPONENT : receives `theme` as a prop ---------- */
export default function MuseumView({ theme }) {
  const [docs,  setDocs]  = useState([]);
  const [stats, setStats] = useState({
    totalDocs: 0, digitizedCount: 0, pendingCount: 0, avgConfidence: 0
  });

  // the custom hook used four times, once per statistic tile
  const animTotal = useCountUp(stats.totalDocs);
  const animDigi  = useCountUp(stats.digitizedCount);
  const animPend  = useCountUp(stats.pendingCount);
  const animConf  = useCountUp(stats.avgConfidence);

  /* SIDE EFFECT : subscribe to Firestore once, unsubscribe on unmount */
  useEffect(() => {
    const q = query(collection(db, "documents"), orderBy("timestamp", "desc"));

    const unsub = onSnapshot(q, (snapshot) => {
      const allDocs = snapshot.docs.map(d => ({ id: d.id, ...d.data() }));
      let digitized = 0, pending = 0, confSum = 0, confCount = 0;

      allDocs.forEach(data => {
        if (data.status === "verified") digitized++; else pending++;
        if (data.ai_output && data.ai_output.confidence) {
          confSum += data.ai_output.confidence;
          confCount++;
        }
      });

      setDocs(allDocs);
      setStats({
        totalDocs:      allDocs.length,
        digitizedCount: digitized,
        pendingCount:   pending,
        avgConfidence:  confCount > 0 ? Math.round((confSum / confCount) * 100) : 0
      });
    });

    return () => unsub();                   // CLEANUP : close the listener
  }, []);                                   // [] -> run once on mount

  const recentFive = docs.slice(0, 5);

  return (
    <div className="max-w-7xl mx-auto px-4 py-12">

      {/* LIST RENDERING with a stable key */}
      <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
        {[
          { label: "Total Documents", value: animTotal },
          { label: "Digitized",       value: animDigi  },
          { label: "Pending Review",  value: animPend  },
          { label: "Avg Confidence",  value: animConf + "%" }
        ].map((stat, i) => (
          <div key={i} className="rounded-xl p-6 text-center border"
               style={{ backgroundColor: theme.surface, borderColor: theme.border }}>
```

```
            <div className="text-3xl">{stat.value}</div>
            <div className="text-xs uppercase">{stat.label}</div>
          </div>
        ))}
      </div>

      <table className="w-full mt-10">
        <thead>
          <tr>
            {["Document ID", "Script", "Status", "Timestamp"].map(h => (
              <th key={h} className="px-6 py-4 text-xs uppercase">{h}</th>
            ))}
          </tr>
        </thead>
        <tbody>
          {/* CONDITIONAL RENDERING : empty state */}
          {recentFive.length === 0 && (
            <tr><td colSpan={4} className="px-6 py-8 text-center">No documents
yet.</td></tr>
          )}

          {recentFive.map((d, i) => (
            <tr key={d.id}>
              <td className="px-6 py-4">{d.id.slice(0, 12)}...</td>
              <td className="px-6 py-4">{d.ai_output?.script || "Unknown"}</td>
              <td className="px-6 py-4">
                {/* TERNARY conditional rendering */}
                <span className={d.status === "verified"
                      ? "text-emerald-400" : "text-amber-400"}>
                  {d.status === "verified" ? "Verified" : "Pending"}
                </span>
              </td>
              <td className="px-6 py-4">
                {d.timestamp?.toDate?.()
                  ? d.timestamp.toDate().toLocaleString()
                  : "In-flight..."}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

Project file - src/components/PublicView.jsx (state, events, conditional rendering)

```
import { useState } from "react";
import { translateText } from "../services/translate";
import { speakText }     from "../services/tts";
import UploadSection     from "./UploadSection";
import MapSection        from "./MapSection";

export default function PublicView({ theme }) {            // PROP in
  // ---------- STATE ----------
  const [image,        setImage]        = useState(null);
  const [base64,       setBase64]       = useState(null);
  const [result,       setResult]       = useState(null);
  const [loading,      setLoading]      = useState(false);
  const [lang,         setLang]         = useState("hi");
  const [translation,  setTranslation]  = useState("");
  const [activeTab,    setActiveTab]    = useState("summary");
  const [isTranslating, setIsTranslating] = useState(false);

  // ---------- EVENT HANDLER : file drop / file pick ----------
  function handleFileDrop(e) {
    e.preventDefault();
    const file = e.dataTransfer ? e.dataTransfer.files[0] : e.target.files[0];
    if (!file) return;
```

```
    setImage(URL.createObjectURL(file));
    const reader = new FileReader();
    reader.onload = () => setBase64(reader.result.split(",")[1]);
    reader.readAsDataURL(file);

    setResult(null);
    setTranslation("");
    setActiveTab("summary");
  }

  // ---------- EVENT HANDLER : key press ----------
  function handleSearch(e) {
    if (e.key === "Enter") { /* query the archive */ }
  }

  return (
    <div className="max-w-7xl mx-auto px-4 py-10">

      {/* CONDITIONAL RENDERING 1 : landing state */}
      {!image && !result && (
        <div className="flex flex-col items-center">
          <h1 className="text-5xl md:text-7xl" style={{ color: theme.accent }}>
            Decode the Past <br />Preserve the Future.
          </h1>
          <div onDragOver={e => e.preventDefault()} onDrop={handleFileDrop}
               onClick={() => document.getElementById("file-input").click()}>
            <h2>Upload Artifact Image</h2>
            <input type="file" id="file-input" accept="image/*"
                   onChange={handleFileDrop} className="hidden" />
          </div>
        </div>
      )}

      {/* CONDITIONAL RENDERING 2 : analysis state */}
      {image && (
        <div className="grid grid-cols-1 lg:grid-cols-12 gap-10">

          {/* LIFTING STATE UP : the parent owns the state, the child reports into it
*/}
          <UploadSection
            image={image}     setImage={setImage}
            base64={base64}   setBase64={setBase64}
            loading={loading} setLoading={setLoading}
            onResult={setResult}
            theme={theme}
          />

          <div className="lg:col-span-7">
            {/* CONDITIONAL RENDERING 3 : skeleton while loading */}
            {loading && <div className="animate-pulse">Decoding...</div>}

            {/* CONDITIONAL RENDERING 4 : tabs */}
            {result && !loading && (
              <>
                <div className="flex gap-6 border-b">
                  {[
                    { id: "summary",     label: "Summary" },
                    { id: "transcript",  label: "Transcript" },
                    { id: "translation", label: "Translation" },
                    { id: "origin",      label: "Origin" }
                  ].map(tab => (
                    <button key={tab.id}
                            onClick={() => setActiveTab(tab.id)}     // EVENT
                            style={{ color: activeTab === tab.id
                                      ? theme.accent : theme.subtext }}>
                      {tab.label}
                    </button>
                  ))}
```

```
                </div>

                {activeTab === 'summary' && <p>{result.summary}</p>}

                {activeTab === 'translation' && (
                  <div>
                    <select value={lang} onChange={e => setLang(e.target.value)}>
                      <option value="hi">Hindi</option>
                      <option value="gu">Gujarati</option>
                      <option value="te">Telugu</option>
                      <option value="en">English</option>
                    </select>

                    <button onClick={async () => {
                      setIsTranslating(true);
                      const t = await translateText(
                        result.modernMarathi || result.transcript, lang);
                      setTranslation(t);
                      setIsTranslating(false);
                    }}>
                      {/* TERNARY conditional rendering */}
                      {isTranslating ? "Processing..." : "Decrypt Metadata"}
                    </button>

                    {translation && (
                      <>
                        <p>{translation}</p>
                        <button onClick={() => speakText(translation)}>Listen</button>
                      </>
                    )}
                  </div>
                )}

                {activeTab === 'origin' && <MapSection locations={result.locations}
                                                       theme={theme} />}
              </>
            )}
          </div>
        </div>
      )}
    </div>
  );
}
```

Project file - src/components/ResultCard.jsx (props + early-return guard)

```
export default function ResultCard({ result, theme }) {
  // CONDITIONAL RENDERING : guard clause, render nothing without data
  if (!result) return null;

  return (
    <div>
      <div className="grid grid-cols-1 sm:grid-cols-2 gap-6 p-6 rounded-xl border"
           style={{ backgroundColor: theme.headerBg, borderColor: theme.border }}>
        <div>
          <div className="text-xs uppercase">Detected Script</div>
          {/* fallback chain : props may be partially filled */}
          <div className="text-2xl">{result.script || result.inferredScript}</div>
        </div>
        <div>
          <div className="text-xs uppercase">Historical Era</div>
          <div className="text-2xl">{result.era || "Unknown"}</div>
        </div>
      </div>

      <p style={{ color: theme.text, borderColor: theme.accent }}>
        {result.summary || "No summary available."}
      </p>
```

```
    </div>
  );
}
```

Output:

- Opening the Museum Admin view animates the four statistic tiles from 0 up to their real values, driven by the custom useCountUp hook.
- Adding a document from another browser tab updates the table instantly, because the useEffect subscription receives the Firestore snapshot without any page refresh.
- Leaving the Museum view runs the cleanup function; the console shows no further snapshot logs, proving the listener was closed.
- In the Public view, dropping an image switches the page from the landing state to the analysis state - this is the {!image && ...} conditional branch flipping.
- Clicking the tab buttons swaps the right-hand panel between Summary, Transcript, Translation and Origin without touching the uploaded image on the left.
- The Decrypt Metadata button label changes to "Processing..." while isTranslating is true, and the Listen button appears only after a translation exists.
- Passing no result to ResultCard renders nothing at all, because of the early-return guard.

Conclusion:

In this assignment, we developed a ReactJS application demonstrating props, state, hooks, event handling and conditional rendering. We used useState for local data, useEffect with a dependency array and a cleanup function for an external subscription, and we wrote a custom hook, useCountUp, that packages reusable animation logic. We handled click, change, drop and key events, rendered output conditionally with &&, the ternary operator and an early return, and lifted state up so that a child component could report its result to its parent.

References:

8. React Official Documentation, https://react.dev/learn
9. React - Built-in Hooks Reference, https://react.dev/reference/react/hooks
10. Vite Guide, https://vite.dev/guide/
11. React - Passing Props to a Component, https://react.dev/learn/passing-props-to-a-component
12. React - Synchronizing with Effects, https://react.dev/learn/synchronizing-with-effects